浙江大学

生成式人工智能课程大作业报告

面向 CODESYS 的 ST 代码生成 Agent

设计、实现与实验评估

课程名称：生成式人工智能

学院专业：控制科学与工程学院 / 待补充

小组成员：待补充（五位成员姓名与学号）

指导教师：待补充

完成日期：2026 年 11 月

成员分工

姓名	学号	主要工作
待补充	待补充	数据集整理与题目审核
待补充	待补充	LTL Spec 与测试用例
待补充	待补充	自研 harness 与 CODESYS 接入
待补充	待补充	基线实验与结果分析
待补充	待补充	报告、PPT 与演示整理

摘　要

PLC Structured Text（ST）代码既要满足语言和工程接口要求，也要在扫描周期中实现正确的控制行为。围绕这一问题，本项目整理了由 20 道公共题和 10 道自建题构成的数据集，逐题编写工具无关的 LTL 规约和共享测试用例，共形成 30 道任务、136 条 LTL 性质、92 个测试用例和 352 个测试步骤。在此基础上，我们实现了一个轻量级 Agent harness，完成任务读取、提示词构建、结构化候选解析、接口预检、CODESYS 编译、编译错误反馈、有限轮修复、Simulation 自动测试和证据保存；同时选择 Codex CLI 作为现成 Agent 基线。实验采用两个框架与两个基座模型组成四种设置，在冻结题目、Spec、模型参数、候选次数和测试标准后完成 120 次正式运行。最终有 90 次首次编译通过、96 次在修复范围内通过编译，205/368 个计划测试通过，53 次运行通过本题全部测试，对应最终编译率 80.00%、测试通过率 55.71% 和任务成功率 44.17%。结果显示，可编译代码与功能正确代码之间仍有明显差距；模型差异大于本实验中的框架差异。项目最终形成了一条从自然语言需求到 ST 候选、CODESYS 执行和可追溯评价的完整流程。

关键词：PLC；Structured Text；Coding Agent；CODESYS；LTL；自动测试

目录

成员分工 I

摘要 I

1 引言：从生成代码到判断代码正确 1

1.1 项目背景 1

1.2 项目目标与主要工作 1

2 数据集、Spec 与测试：建立统一的正确性标准 2
2.1 数据集构成 . . . 2
2.2 公共题转换与工程适配 . . . 2
2.3 自建题的来源、改编与合理性 . . . 2
2.4 从自然语言到 LTL Spec . . . 3
2.5 共享测试用例 . . . 4

3 自研 Agent harness：从最小闭环到最终方案 4
3.1 最初实现与实际问题 . . . 4
3.2 问题驱动的改进 . . . 5
3.3 最终架构与职责边界 . . . 5
3.4 生成、编译、修复与终止 . . . 6
3.5 Codex CLI 基线与公平接入 . . . 6

4 实验环境与公平实验设计 7
4.1 实验环境 . . . 7
4.2 模型、参数与四种设置 . . . 7
4.3 公平性控制 . . . 7
4.4 评价指标 . . . 8

5 最终实验结果 8
5.1 总体结果 . . . 8
5.2 四种设置对比 . . . 9
5.3 模型、框架、难度与题库类型 . . . 9
5.4 初始完整实验与最终实现 . . . 11

6 结果分析与代表案例 12
6.1 编译通过与功能正确之间的差距 . . . 12
6.2 模型差异 . . . 12
6.3 框架差异 . . . 13
6.4 成功案例：`public_20` . . . 13
6.5 语义失败案例：`custom_06` . . . 13
6.6 组合差异案例：`public_19` . . . 13
6.7 失败类型与局限 . . . 14

7 结论 **14**

A 电子附录与证据目录 **14**

B 报告数字追溯表 **14**

1 引言：从生成代码到判断代码正确

1.1 项目背景

ST 是 IEC 61131-3 标准中的文本化 PLC 编程语言，适合表达状态保持、互锁、顺序控制、定时和数据处理等逻辑。大模型已经能够根据自然语言生成形式完整的程序，但 PLC 任务与普通算法题存在明显差异：程序会被周期性扫描，输出常常依赖此前状态，停止、急停和故障还具有明确优先级。模型给出代码之后，仍需要真实工程环境确认接口、类型和调用链，再通过功能测试检查控制行为。

课程中关于软件测试的讨论指出，若测试只覆盖少数样例，即使全部通过，程序仍可能在未覆盖输入上出错。这个结论直接影响了本项目的评价方法。我们将问题分成三个层次：候选结构是否符合题目接口，代码能否通过 CODESYS 编译，运行行为能否通过需求对应的测试。后文因此分别报告编译率、测试通过率和任务成功率。

1.2 项目目标与主要工作

课程作业要求在全部公共题和不少于 10 道自建题上，对自研框架、现成 Agent 和两个基座模型进行四种设置的公平比较。围绕这一目标，项目完成了四项相互衔接的工作：

1. 将公共题和自建题统一为可执行的数据结构，并记录自建题的来源、改编、审核和 PLC 合理性；
2. 逐题编写 LTL Spec 与测试用例，使自然语言需求可以追踪到具体性质和测试步骤；
3. 打通模型、Python、CODESYS 编译与 Simulation 自动测试，建立生成、检查、修复和保存证据的闭环；
4. 冻结实验条件后完成 120 次正式运行，从模型、框架、难度和题库类型等角度分析结果。

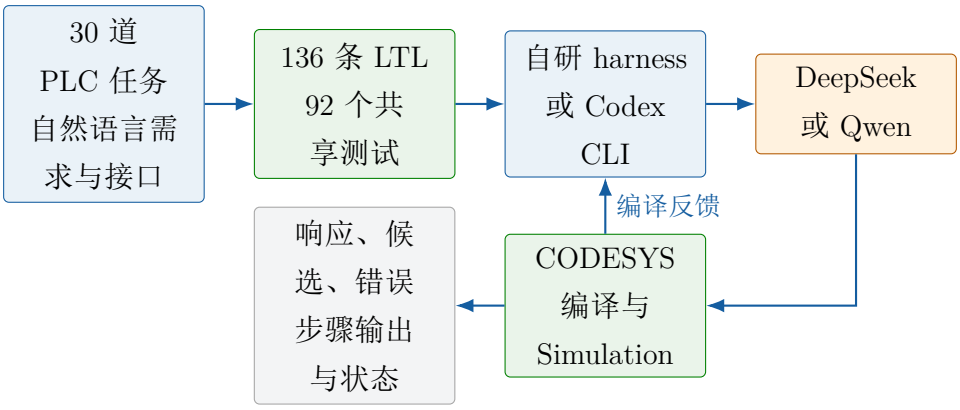


图 1 项目从需求到实验结果的总体关系

图 1 展示了项目主线。题目与 Spec 定义输入和约束，Agent 与模型产生候选，CODESYS 提供真实编译和测试结果，所有中间产物进入原始证据目录。报告中的数字均由这些记录重新汇总生成。

2 数据集、Spec 与测试：建立统一的正确性标准

2.1 数据集构成

项目数据集包含老师提供的 20 道公共题和 10 道自建题。公共题覆盖基础控制、报警、状态机、数据结构和复杂生产流程；自建题补充了安全锁存、报警复位、批次流程、定时监控和现场联锁。最终规模见表 1。

表 1 冻结数据集规模

类型	题目数	LTL 性质	测试用例	测试步骤
公共题	20	92	62	263
自建题	10	44	30	89
合计	30	136	92	352

30 道题按统一标准分为 easy 9 道、medium 12 道和 hard 9 道。难度由状态数量、时序依赖、数据结构、模式数量和安全约束综合判断，主要用于观察不同任务组成下的表现，不把它视为严格的理论复杂度量。

2.2 公共题转换与工程适配

老师提供的原始材料保持只读，实验使用转换后的 JSON。每道题明确任务 ID、需求、POU 名称与类型、公开接口、难度和必要的编译上下文。转换没有改写控制目标，主要解决原始题面与自动实验之间的接口衔接。

其中，使用 DUT、数组和 `VAR_IN_OUT` 的题目需要在临时工程中创建固定类型并绑定实例；`public_18` 的 FIFO 容量统一为 10，元素类型统一为 DINT。四种设置读取完全相同的转换结果，因此这些工程适配不会改变组间比较条件。

2.3 自建题的来源、改编与合理性

10 道自建题由自主构思 2 道、经验改编 6 道和规则合成 2 道组成，难度分布为 easy 3 道、medium 4 道、hard 3 道。每道题都保存了来源类型、创建或合成过程、人工修改、审核记录、PLC 合理性和多样性标签。

题目构建遵循相同过程：先从现场控制场景提取一个边界清晰的功能块任务，再确

定公开输入输出和初始状态，补充急停、故障、复位、保持或最小时间等规则，最后由题面、Spec 和测试交叉审核。例如，custom_01 强调急停锁存与人工复位，custom_06 包含填充、混合、排空的批次状态推进，custom_09 关注输送机在时间窗口内的堵料监测。这些任务能够用 PLC 扫描周期和现场信号自然表达，也覆盖了从布尔控制到多阶段时序逻辑的变化。

表 2 代表性自建题及审核重点

题目	场景	难点	人工审核重点
custom_01	设备安全锁存	安全优先级	急停解除后不得自动恢复，必须人工复位
custom_06	配料批次流程	多阶段状态机	填充、混合、排空顺序与阶段退出条件完整
custom_09	输送机堵料监控	定时与复位	连续异常达到时间窗口才报警，恢复条件明确

2.4 从自然语言到 LTL Spec

自然语言题面便于阅读，但安全优先级、保持条件和扫描后的状态变化容易产生不同理解。为此，每道题先定义命题，再按安全、响应、保持、优先级和不变量等类型写成工具无关 LTL。项目只使用了与任务直接相关的基本算子：G 表示性质始终成立，X 表示当前输入经过一次 PLC 扫描后的状态。

以高低限报警题 public_03 为例，要求高报警越过上限时置位，只有低于上限减滞回窗口后才清除；配置错误还要优先返回对应状态码。表 3 给出需求、性质与测试的对应关系。

表 3 `public_03` 的需求—LTL—测试对应示例

需求	LTL 性质	测试覆盖
上下限配置非法时优先报错	$G(invalid_levels \rightarrow X(error \wedge status_8001))$	配置错误优先级
越过高限后触发高报警	$G((valid \wedge above_hi) \rightarrow X(hi_alarm))$	高报警置位
进入清除区后清除报警	$G((valid \wedge below_hi_clear) \rightarrow X(\neg hi_alarm))$	滞回保持与清除
合法配置返回成功状态	$G(valid \rightarrow X(\neg error \wedge status_ok))$	正常配置状态码

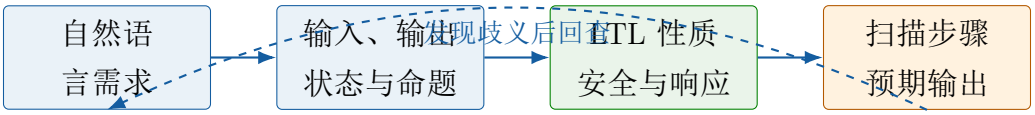


图 2 需求、LTL 与测试之间的追踪关系

2.5 共享测试用例

每个测试用例包含初始状态、输入步骤、等待时间、预期输出和覆盖内容。执行器只通过公开输入建立状态，不直接改写候选内部变量；每一步写入输入后等待 PLC 扫描，再读取输出并比较。正常流程、冲突输入、安全优先级、状态保持、复位和时间边界均在题目需要时纳入测试。

同一道题的四种设置使用同一份测试和判定标准。测试内容不提供给 Agent，避免候选针对答案进行拟合。编译失败时，本题计划测试全部计为未通过。LTL 在本项目中用于整理需求并进入最终框架的任务上下文；项目没有接入 PLCVerif，因此不报告形式化验证通过率。

3 自研 Agent harness：从最小闭环到最终方案

3.1 最初实现与实际问题

项目先实现最小闭环：读取一道题，调用模型生成 ST，写入临时工程，启动 CODESYS 编译；若编译失败，则把错误反馈给模型，最多修复两次。这一步确认了模型 API、候选格式和 CODESYS 外部边界能够接通，也暴露出三个问题。

第一，部分候选的 POU 名称、类型或公开接口明显错误，仍会消耗一次 CODESYS 启动。第二，自然语言题面对保持、优先级和状态变化的表达不够稳定，编译通过后的功能失败较多。第三，编译与测试分别启动 CODESYS，而测试脚本本身还要重新编译工程，形成重复开销。

3.2 问题驱动的改进

针对上述问题，最终实现增加了三个直接改动。候选进入 CODESYS 前先检查 POU 名称、类型、声明头和公开接口；逐题 LTL Spec 与自然语言需求一起提供给两个框架；编译与共享测试合并为一次 CODESYS 执行。测试结果仍只用于最终评价，不反馈给模型。

表 4 从问题到最终设计的对应关系

开发中观察到的问题	修改	设计目的
明显接口错误仍启动外部工具	增加本地候选预检	提前结束确定无效的候选
编译通过后存在语义错误	将逐题 LTL 加入共享上下文	明确安全、保持和状态变化要求
编译和测试重复启动 CODESYS	合并为一次执行	减少外部进程启动和重复工程加载
测试可能泄露预期行为	测试结果不进入修复上下文	保持评价独立与四组公平

3.3 最终架构与职责边界

最终 harness 保持少量模块和直接数据流。任务加载器读取冻结题目与 Spec；提示词模块生成首次请求或修复请求；模型客户端只负责调用 API；候选解析器提取结构化字段并执行接口预检；CODESYS 执行器创建临时工程、编译并运行测试；运行器控制候选次数、终止条件和证据目录；汇总脚本从原始记录生成 CSV、JSON、表格和图片。

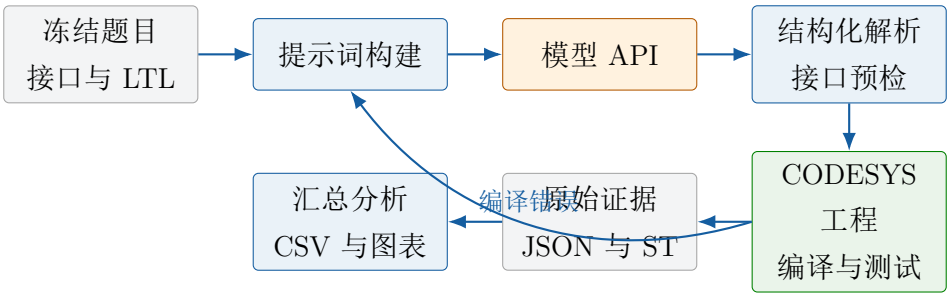


图 3　最终 harness 架构与数据流

3.4　生成、编译、修复与终止

初次请求包含需求、公开接口、必要编译上下文和 LTL Spec。模型按固定 JSON 格式返回 `pou_name`、`pou_type`、`declaration` 和 `implementation`。解析成功且接口预检通过后，候选功能块会在临时 `PLC_PRG` 中实例化并调用，确保代码真正进入编译链。

若编译失败，系统提取错误对象、位置和文本，下一轮只增加上一候选与编译错误。每题最多产生三个候选；编译成功后直接进入共享测试，测试结束即终止。候选次数达到上限或模型调用出现不可恢复错误时也会终止，并保存根因。

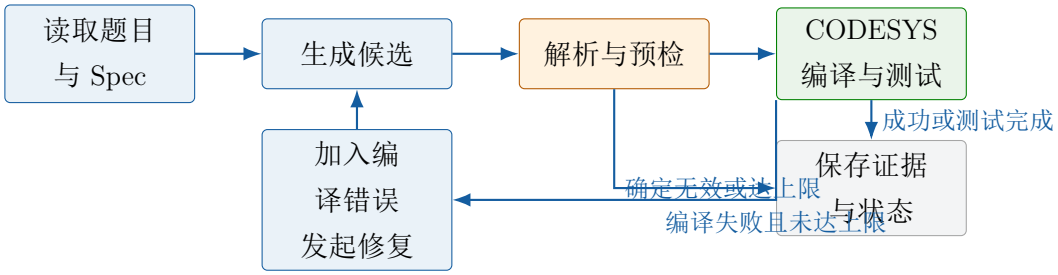


图 4　生成、检查、修复和终止流程

这一流程与课程中的 propose–check–repair 思路一致：模型提出候选，本地预检与 CODESYS 完成检查，编译错误驱动有限轮修复。功能测试作为独立评价门，在生成过程结束后给出最终判定。

3.5　Codex CLI 基线与公平接入

现成 Agent 基线采用 Codex CLI。它与自研 harness 使用相同模型、题目、Spec、候选上限和 CODESYS 执行器。Codex CLI 负责产生结构化候选，外部 Python 运行器把上一候选和编译错误交给下一轮。两个框架最终都输出同一数据合同，因此测试与汇总阶段不需要分叉。

4 实验环境与公平实验设计

4.1 实验环境

正式实验在同一台计算机上顺序执行，环境见表 5。模型通过 DMXAPI 接入，PLC 代码在 CODESYS Simulation 中运行。当前 Codex CLI 的冻结版本号来自正式实验配置，报告写作环境中检测到的桌面内置可执行文件不用于替换该记录。

表 5 正式实验环境

项目	配置
操作系统	Microsoft Windows 11 家庭中文版，10.0.26200
处理器与内存	AMD Ryzen 7 7840H，15.2 GB 可用物理内存
Python	3.12.4
PLC 开发环境	CODESYS V3.5 SP22 Patch 4，Simulation
现成 Agent	Codex CLI 0.167.0-alpha.3.2
模型接入	DMXAPI，HTTP API
自研部分	Python harness、CODESYS 自动化脚本、汇总与绘图脚本

4.2 模型、参数与四种设置

模型 A 为 `deepseek-v4.1-flash-1017`，模型 B 为 `qwen3-coder-flash`。两个模型均使用服务默认推理配置，不额外传入 `reasoning_effort`。单次请求超时为 300 秒，每题允许一次首次生成和最多两次修复。自动测试默认等待 50 ms，以覆盖 PLC 扫描。

表 6 四种正式实验设置

Agent 框架	DeepSeek	Qwen
Codex CLI	30 道题	30 道题
自研 harness	30 道题	30 道题

4.3 公平性控制

输入阶段，同一道题使用相同需求、公开接口、编译上下文和 LTL Spec；执行阶段，两个框架使用相同模型、候选次数、超时和 CODESYS 执行器；判定阶段，四组使用相同测试，测试内容不向 Agent 开放。模型质量失败、编译失败和功能测试失败按真实结果保留，只有原始记录能证明 SSL、连接重试或执行器故障时才允许重跑。

每次运行都保存配置快照、提示词、模型原始响应、候选 ST、编译消息、测试步骤输出和最终状态。由此形成“原始记录—逐任务汇总—四组汇总—报告图表”的证据链。

4.4 评价指标

首次编译通过率衡量第一个候选的可编译性；最终编译通过率衡量三个候选范围内的可编译性；测试通过率以每组固定的 92 个计划测试为分母；任务成功要求候选通过编译且本题全部测试通过。运行时间从任务开始到最终状态，Token 与已知费用直接读取接口记录。

5 最终实验结果

5.1 总体结果

120 次正式运行全部产生有效终态，没有模型错误、请求超时或运行器错误。90 次首次候选编译通过，修复后共有 96 次通过编译；这 96 次中，53 次通过本题全部测试，另外 43 次存在至少一个功能测试失败。总体测试通过数为 205/368。

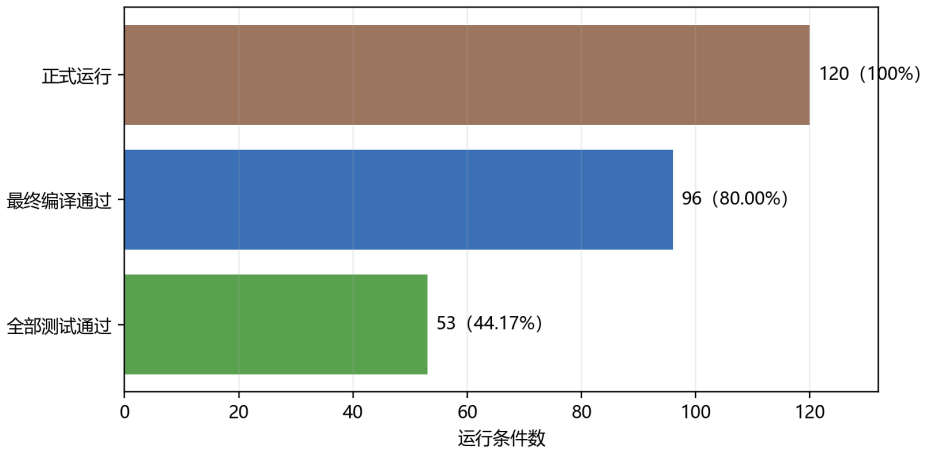


图 5 120 次正式运行的结果漏斗

由图 5 可得首次编译率为 75.00%，最终编译率为 80.00%，测试通过率为 55.71%，任务成功率为 44.17%。最终编译率与任务成功率相差 35.83 个百分点，这个差距构成后续分析的核心。

5.2 四种设置对比

表 7 四种设置的正式实验结果

框架	模型	首次编译	最终编译	测试通过	任务成功	平均修复	平均耗时/s
Codex CLI	DeepSeek	96.67%	100.00%	77.17%	63.33%	0.033	129.37
Codex CLI	Qwen	50.00%	56.67%	39.13%	30.00%	0.967	129.43
自研 harness	DeepSeek	93.33%	100.00%	69.57%	53.33%	0.067	119.52
自研 harness	Qwen	60.00%	63.33%	36.96%	30.00%	0.767	80.02

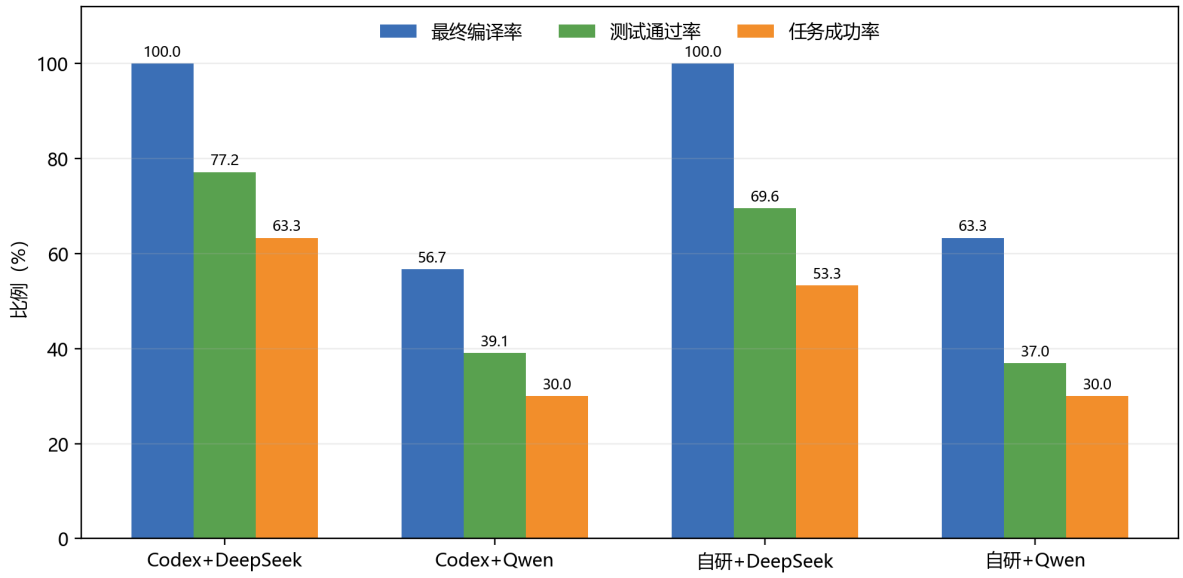


图 6 四种设置的最终编译率、测试通过率与任务成功率

Codex CLI + DeepSeek 的三项核心指标最高，任务成功率为 63.33%；自研 harness + DeepSeek 的任务成功率为 53.33%。两个 Qwen 设置的任务成功率均为 30.00%，但自研 harness + Qwen 的最终编译率略高，平均耗时较低。

5.3 模型、框架、难度与题库类型

合并框架后，DeepSeek 在 60 次运行中成功 35 次，任务成功率为 58.33%，测试通过率为 73.37%；Qwen 成功 18 次，任务成功率为 30.00%，测试通过率为 38.04%。合并模型后，Codex CLI 成功 28/60，自研 harness 成功 25/60，对应 46.67% 和 41.67%。模型之间的差距明显大于框架之间的总体差距。

表 8 按难度和题库类型汇总的结果

分组	任务成功率	测试通过率
easy	61.11%	61.11%
medium	33.33%	51.39%
hard	41.67%	56.03%
公共题	36.25%	48.79%
自建题	60.00%	70.00%

hard 在本次实验中高于 medium，这与具体题目组成和需求表达有关，不能推广为一般规律。自建题的需求边界、接口和审核记录更统一，其总体结果高于公共题。



图 7　30 道题在四种设置下的终态（绿：成功；橙：测试失败；红：编译失败）

热力图显示，`custom_01`、`custom_02`、`custom_05`、`public_01`、`public_03` 和 `public_20` 在四组中均成功；另一些题目在模型或框架变化后出现不同结果，适合用于案例分析。

5.4　初始完整实验与最终实现

为了观察项目改进带来的整体变化，我们比较了两个覆盖相同 120 个条件的完整实验。该比较同时包含上下文、预检和执行方式等系统改进，用于展示迭代结果，不作为单变量消融。

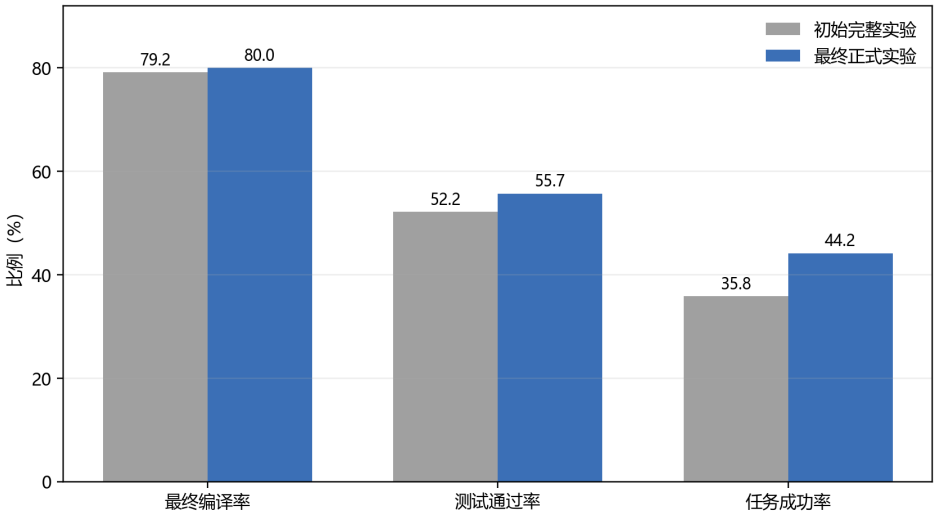


图 8　初始完整实验与最终正式实验的核心指标

任务成功数由 43 增加到 53，测试通过数由 192 增加到 205，最终编译数由 95 增加到 96。对应地，任务成功率提高 8.34 个百分点，测试通过率提高 3.54 个百分点。配对比较中有 19 个条件的终态改善，92 个不变，9 个下降。

合并编译与测试后，原始目录中的 CODESYS 编译/测试进程记录由 271 次减少到 170 次，下降 37.27%。端到端平均任务耗时由 111.25 s 增至 114.59 s，增加约 3.00%。外部进程启动减少，但模型响应波动、候选差异和更完整的上下文共同影响端到端时间，因此本次优化主要改善执行结构，没有形成稳定的总耗时下降。

6　结果分析与代表案例

6.1　编译通过与功能正确之间的差距

96 个条件通过最终编译，其中 43 个没有通过本题全部测试。这些候选已满足 ST 语法、类型、接口和工程调用链，却在状态保持、优先级、定时或阶段推进上与需求不一致。编译器解决的是程序能否进入 PLC 工程，功能测试回答程序在指定场景中做得是否正确。两类证据共同使用，才能判断任务是否完成。

测试通过率 55.71% 高于任务成功率 44.17%，也说明不少失败任务并非全部逻辑错误，而是在部分场景中出现偏差。逐步骤记录能够定位第一个不一致的扫描周期，为后续分析提供依据。

6.2　模型差异

DeepSeek 在两个框架下均达到 100% 最终编译率，Qwen 合并后的最终编译率为 60%。两者的测试通过率分别为 73.37% 和 38.04%。从逐题结果看，Qwen 在复杂接口、多模式状态推进和较长上下文下更容易出现编译或功能问题。因此，在本实验的数据集

和参数范围内，基座模型能力是影响最终结果的主要因素之一。

Token 记录也呈现出不同调用特征。Codex CLI + Qwen 的总 Token 为 3,248,894，明显高于其他三组；自研 harness 的两个模型合计 Token 较少。由于 Codex CLI 没有可直接比较的费用字段，本报告保留 Token 事实，不估算其费用。

6.3 框架差异

Codex CLI 的总体任务成功率为 46.67%，自研 harness 为 41.67%，相差 5 个百分点。DeepSeek 下 Codex CLI 高 10 个百分点，Qwen 下两者任务成功率相同；自研 harness + Qwen 的最终编译率高 6.66 个百分点。由此看，框架影响会随模型和题目变化，单一总体排名无法覆盖全部情况。

自研 harness 的价值还体现在实验控制能力：任务上下文、候选格式、修复上限、外部工具调用、测试隔离和证据目录都有明确边界。它没有依赖多 Agent 编排或复杂插件系统，主要模块可以直接追踪到课程要求中的生成、编译、修复、测试和公平比较。

6.4 成功案例：public_20

`public_20` 是五工位生产控制任务，难度为 hard，包含多工位状态和协同控制。四种设置均通过编译并通过 4/4 测试。该结果说明，当题目的模式边界、接口和状态推进足够明确时，不同框架和模型都可能形成可执行的完整控制逻辑。

这个案例也解释了 hard 组结果没有低于 medium 的一部分原因：难度标签描述任务包含的结构，但题面表达质量和测试路径同样影响模型表现。`public_20` 虽然状态较多，流程关系相对清晰。

6.5 语义失败案例：custom_06

`custom_06` 描述填充、混合、排空三个阶段。四种设置全部通过编译，但没有一组通过全部测试；四组共通过 6/12 个测试。候选能够建立基本状态机，却会在阶段退出、计时完成或复位后的状态上产生偏差。

这个案例是编译通过不等于功能正确的直接证据。若实验只记录编译结果，四组都会被判为成功；加入共享测试后，才能发现多阶段任务中的控制语义问题。

6.6 组合差异案例：public_19

`public_19` 包含手动、单步、单周期和连续模式。四种设置都通过编译，只有 Codex CLI + DeepSeek 通过全部 4 个测试，其余三组存在功能失败。该题需要同时处理模式互斥、状态推进和复位，模型与框架组合会影响上下文理解和最终控制结构。

6.7 失败类型与局限

从代表性记录中可以归纳出候选格式或公开接口错误、数据类型与数组绑定错误、状态保持遗漏、停止或故障优先级错误、多阶段跳转错误、定时与复位边界错误。项目没有对全部失败候选进行人工标签，因此不绘制失败类型比例，避免将案例观察误写为完整统计。

本实验仍有五项局限。第一，正式结果来自一台计算机和一个 CODESYS 版本。第二，每个条件正式运行一次，无法估计模型输出方差。第三，初始到最终实现同时修改多个部分，完整实验对比不能分离单项贡献。第四，LTL 用于需求组织和上下文，尚未接入 PLCVerif。第五，92 个测试覆盖主要行为，仍不能穷尽所有输入和状态。

7 结论

本项目围绕 PLC ST 代码生成建立了完整、可执行的实验链路。数据部分包含 30 道题、136 条 LTL 性质、92 个测试用例和 352 个测试步骤；系统部分实现了自研 harness、Codex CLI 基线、两个模型接入、CODESYS 编译和 Simulation 自动测试；实验部分完成了四种设置共 120 次正式运行，并保存从原始响应到逐步骤测试结果的证据。

最终实验的首次编译率为 75.00%，最终编译率为 80.00%，测试通过率为 55.71%，任务成功率为 44.17%。DeepSeek 的总体表现明显高于 Qwen，两个框架的总体差距相对较小。96 个可编译条件中有 43 个仍存在功能失败，说明 PLC 代码生成的主要难点已经从形式完整延伸到状态、优先级和时序语义。

项目改进围绕真实问题逐步展开：本地预检减少无效外部调用，LTL 上下文补充语义约束，编译与测试合并减少重复 CODESYS 启动。最终方案保持了清晰的数据流和可追溯结果。后续可将 LTL 接入 PLCVerif，对同一条件进行多次重复实验，并研究在不泄露测试答案的前提下利用功能反馈修复代码。

A 电子附录与证据目录

完整题目位于 `data/tasks/`，逐题 LTL 位于 `data/specs/`，共享测试位于 `data/test_cases/`。最终正式实验原始证据位于 `experiments/raw/formal/20261111_102220_v4_formal/`；报告使用的逐任务汇总、四组汇总和初始—最终对比分别保存在 `experiments/processed/` 下。本文中的表格和图片由 `experiments/analysis/` 中的脚本从上述 JSON 生成。

B 报告数字追溯表

报告内容	处理后数据	原始来源
四种设置结果	`formal_summary.json/csv`	最终正式批次 120 个运行目录
逐题终态与耗时	`formal_runs.json/csv`	每个条件的 `run_result.json`
初始与最终对比	`initial_final_comparison.json/csv`	两个完整正式批次
报告分析表	`formal_analysis.md`	汇总脚本自动生成
图 6、7、8	`generate_report_figures.py`	处理后 JSON